AI Food and Macro Tracker – V1 Design Document

Version 1.1 | 23 October 2026 | Builders: Vishal and Riya | Platform: Android first (Flutter)

This document covers the design phase of the SDLC for Version 1: scope, architecture, data, API, module design, UI/UX, flows, security, error handling, test design, deployment and traceability. Where it differs from the Project Overview v1.0 (17 October), this document wins.

1. Purpose and scope

The app lets a user photograph a meal, get an AI estimate of its calories and macros, save it, and track daily calories against a target. V1 is deliberately small: five features, one backend function, one local database.

1.1 V1 features

ID	Feature	Summary
F1	Account	Sign up, log in, stay logged in. Collect profile details (goal, age, gender, height, weight, activity level) and calculate targets.
F2	Meal photo analysis	User takes a photo of a meal. A server function sends it to an LLM vision API and returns the nutrition estimate.
F3	Result details	Show each food item with grams, calories, protein, carbs, fat, labelled as AI estimate. User can adjust portion, then save.
F4	Daily scan limit	Each account has a fixed number of analyses per day. Remaining count is always visible.
F5	History, chart and alerts	Entries are stored on the device for 1, 3 or 6 months (user's choice). Daily calorie totals are charted against the target. Warnings appear near and over the limit.

1.2 Deferred to V2

Barcode scanning, saved meals and custom foods, cloud sync and backup, payments, home-screen widget, iOS, micronutrients, exports, social features.

1.3 Changes from Overview v1.0

Topic	Overview v1.0	This document	Reason
Accounts	Out of scope	In scope (F1)	Needed to enforce the scan limit per person, which also settles the open decision about limits surviving reinstalls
Photo capture	Live auto-capture	Camera screen with a shutter button	Simpler, more reliable on low-end phones, matches "click the picture"
Barcode	In scope	V2	Keeps V1 small
History chart	Later	In scope (F5)	Requested for V1
History length	Fixed	User chooses 1, 3 or 6 months	Requested for V1

2. Key design decisions

#	Decision	Choice	Reason
D1	App framework	Flutter (Dart)	Fast to build, mature camera support, small enough, iOS later from the same code
D2	Authentication	Firebase Authentication: email and password, plus Google sign-in as a Should	No auth server to build. The session persists, so users stay logged in.
D3	Backend	One Firebase Cloud Function, `analyzeMeal`	Auth is built in (the function receives a verified user id), keeps the AI key off the phone, no server to manage

#	Decision	Choice	Reason
D4	AI	A vision-capable LLM through its API (Gemini planned), called only from the function	One call returns structured JSON for the whole plate
D5	Where history lives	SQLite on the phone (Drift)	Requested: history stays on the user's device. Works offline.
D6	What the server stores	Only the scan counter per user per day	No meal data and no photos on any server
D7	Scan-limit day	Resets at midnight India time (IST), decided by the server	A phone clock cannot be used to cheat the limit
D8	Daily limit value	5 scans per day, stored as a server setting	Tune after testing with real users
D9	Calorie day	Resets at the phone's local midnight	Matches what the user experiences
D10	Data per account on device	One database file per user id	Switching accounts never mixes data

3. Requirements used for design

3.1 Functional requirements

Priority: M = Must, S = Should.

ID	Requirement	Feature	Pri
FR-01	User can sign up with name, email and password	F1	M
FR-02	User can log in and log out	F1	M
FR-03	Session persists across app restarts until logout	F1	M
FR-04	Password reset by email	F1	S
FR-05	Google sign-in	F1	S
FR-06	After first login, collect goal, age, gender, height, weight, activity level, with range checks (age 16 to 90, height 100 to 230 cm, weight 30 to 250 kg)	F1	M

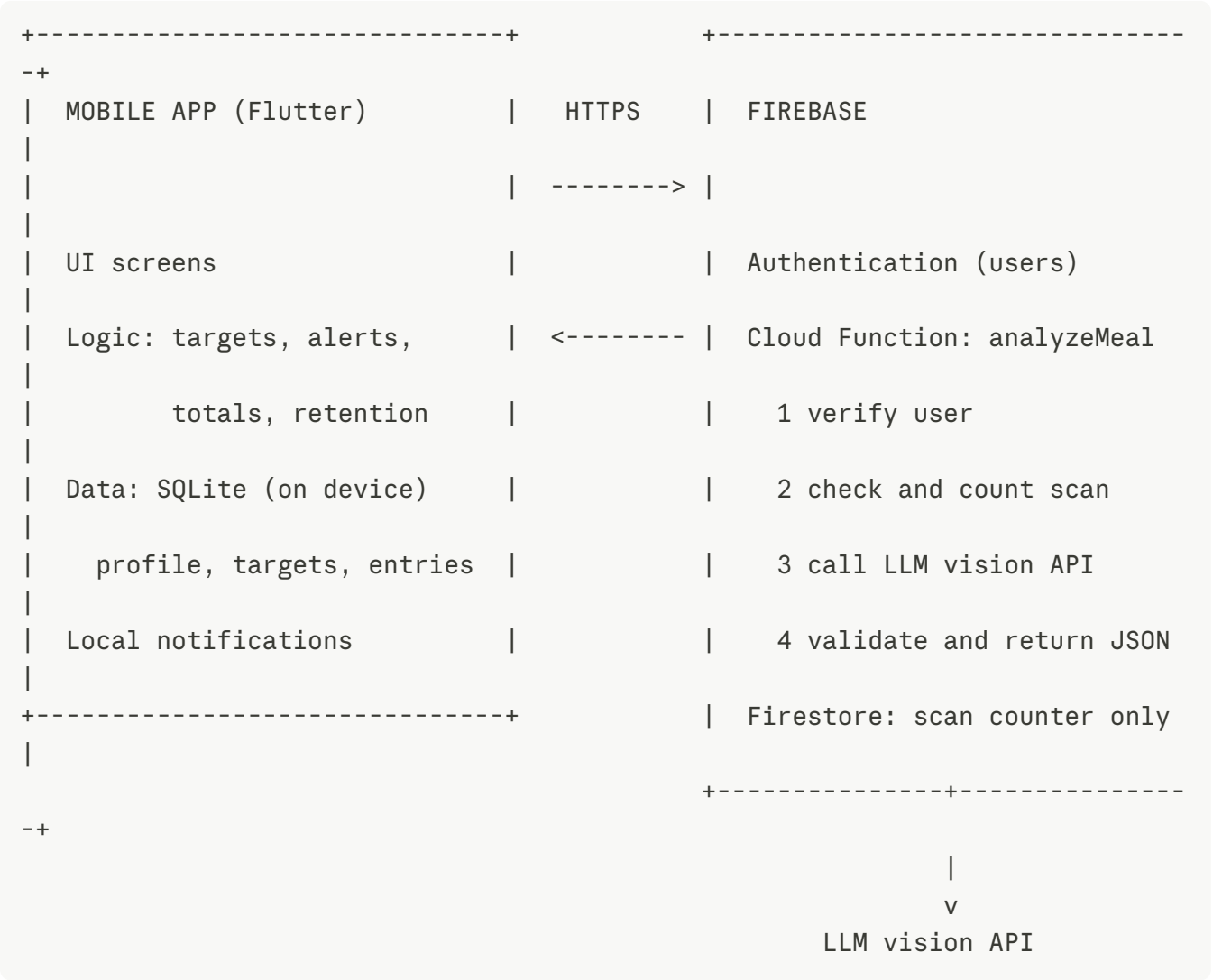
ID	Requirement	Feature	Pri
FR-07	Calculate daily calories and macros from the profile; user can override any value	F1	M
FR-08	Show a not-medical-advice note at setup and in Settings	F1	M
FR-09	Camera screen with permission explanation and a shutter button; gallery pick as fallback	F2	M
FR-10	Compress the photo before upload	F2	M
FR-11	Send the photo to the backend, which calls the LLM and returns structured nutrition JSON	F2	M
FR-12	Show a loading state and handle failure with retry	F2	M
FR-13	Result card lists every detected item with grams, kcal, protein, carbs, fat and the AI-estimate label	F3	M
FR-14	User can rename an item, change grams or remove an item; nutrition recalculates proportionally	F3	M
FR-15	Save to today writes the entries to the local database in one tap	F3	M
FR-16	Manual quick add (name, kcal, protein, carbs, fat) as a fallback when scans are unavailable	F3	S
FR-17	Server enforces the daily scan limit per user	F4	M
FR-18	App shows scans remaining, and a clear message when the limit is reached	F4	M
FR-19	Today's totals are the sum of today's entries; a new day starts at zero	F5	M
FR-20	Home shows calories eaten, calories left, and protein, carbs and fat against targets	F5	M
FR-21	User can edit or delete an entry; totals update immediately	F5	M
FR-22	User chooses a retention period of 1, 3 or 6 months in Settings (default 3); older entries are deleted automatically	F5	M
FR-23	History screen shows a daily calorie chart with the target line for 7 days, 30 days, or the full retention period	F5	M

ID	Requirement	Feature	Pri
FR-24	Tapping a day shows its entries and totals	F5	M
FR-25	In-app warning at 80% of the calorie target and a red over-limit state above 100%	F5	M
FR-26	Local notification when a threshold is crossed, once per threshold per day; user can turn it off	F5	S
FR-27	Optional evening reminder if protein is clearly below target	F5	S
FR-28	Delete all local data and delete account from Settings	F1	S

3.2 Non-functional requirements

ID	Requirement
NFR-01	Photo analysis returns within about 5 seconds for 90% of requests on a normal mobile connection
NFR-02	App download size at most about 25 MB
NFR-03	Everything except login, sign-up and photo analysis works offline
NFR-04	Runs on Android 10 to 14, including low-end phones
NFR-05	All network traffic is encrypted; no secret keys inside the app
NFR-06	Photos are never stored on a server and are not kept on the phone in V1
NFR-07	Text is readable (contrast at least 4.5:1) and tap targets are at least 44 px

4. Architecture (high level)



4.1 Responsibilities

Part	Owns
Mobile app	All screens, calculations, history, chart, alerts, notifications, local database
Firebase Authentication	Accounts, passwords, sessions and tokens
Cloud Function	The AI key, the scan limit, calling the LLM, validating the answer
Firestore	One small document per user per day holding the scan count
LLM API	Turning a meal photo into items and nutrition estimates

4.2 What is stored where

Data	Location
Email, password hash, user id	Firebase Authentication
Scan count per day	Firestore
Profile, targets, settings	Phone (SQLite)
Meal entries and history	Phone (SQLite)
Photos	Not stored. Processed in memory and discarded.

Known limitation of V1: because history is only on the phone, logging in on a new phone or reinstalling starts with empty history. Cloud backup is a V2 item.

5. Technology stack

Layer	Choice
App	Flutter, Dart
State management	Riverpod
Local database	Drift (SQLite)
Camera	`camera` and `image_picker` packages
Image compression	`flutter_image_compress`
Charts	`fl_chart`
Notifications	`flutter_local_notifications`
Auth	`firebase_auth` (and `google_sign_in` for FR-05)
Backend call	`cloud_functions`
Backend	Firebase Cloud Functions (Node.js)
Backend data	Firestore
AI	LLM vision API, key stored as a function secret
Source control	Git, one branch per feature, review before merge

Cloud Functions need the Firebase pay-as-you-go plan to call an outside API. At this scale the cost is very small, but set a budget alert.

6. Data design

6.1 Local database (SQLite, one file per user id)

profile (one row)

Field	Type	Notes
id	int	always 1
name	text	
goal	text	cut, maintain, bulk
age	int	16 to 90
gender	text	male, female
height_cm	real	100 to 230
weight_kg	real	30 to 250
activity	text	sedentary, light, moderate, active, very_active
retention_months	int	1, 3 or 6. Default 3.
notifications_on	bool	default true
reminder_time	text	HH:MM, optional

targets (one row)

Field	Type	Notes
id	int	always 1
calories	int	
protein_g	int	
carbs_g	int	
fat_g	int	
is_custom	bool	true once the user edits any value

meal_entry

Field	Type	Notes
id	int	primary key
entry_date	text	local date, YYYY-MM-DD, indexed
logged_at	text	ISO timestamp
meal_type	text	breakfast, lunch, snack, dinner
name	text	
grams	real	
kcal	real	
protein_g	real	
carbs_g	real	
fat_g	real	
source	text	ai or manual

alert_log (to send each notification once per day)

Field	Type	Notes
alert_date	text	YYYY-MM-DD
level	text	warning, over, protein_reminder

Primary key is (alert_date, level).

Rules:

- Nutrition values are copied into each `meal_entry` row, so history stays correct if anything changes later.
- Daily totals are computed, not stored: `SELECT entry_date, SUM(kcal), SUM(protein_g), SUM(carbs_g), SUM(fat_g) FROM meal_entry GROUP BY entry_date`.
- The chart uses the same query for the chosen date range.

6.2 Retention rule

On every app start, and again after midnight while the app is open:

1. cutoff = today minus retention_months

2. delete from `meal_entry` where `entry_date` is before cutoff
3. delete from `alert_log` where `alert_date` is before cutoff

When the user shortens the period (for example 6 months to 1 month), show a confirmation: "Entries older than 1 month will be deleted." Lengthening the period never restores deleted data.

6.3 Server data (Firestore)

Collection `scanUsage`, document id `{uid}_{YYYY-MM-DD in IST}`:

Field	Type
uid	string
date	string
count	number

Collection `config`, document `limits`: `dailyScanLimit` (number, default 5).

Client access to Firestore is blocked by security rules. Only the Cloud Function reads and writes it.

7. Business logic design

7.1 Daily targets

Basal metabolic rate (Mifflin-St Jeor):

- Male: 10 x weight_kg + 6.25 x height_cm - 5 x age + 5
- Female: 10 x weight_kg + 6.25 x height_cm - 5 x age - 161

Activity factors: sedentary 1.2, light 1.375, moderate 1.55, active 1.725, very active 1.9.

Calories = BMR x factor, then goal: cut x 0.85, maintain x 1.0, bulk x 1.10. Protein = 1.8 g x weight_kg. Fat = 25% of calories / 9. Carbs = (calories - 4 x protein - 9 x fat) / 4.

Worked example (also test TC-01): male, 21, 175 cm, 70 kg, moderate, maintain. BMR = 700 + 1093.75 - 105 + 5 = 1693.75. Calories = 1693.75 x 1.55 = about 2,625. Protein 126 g, fat about 73 g, carbs about 366 g.

Values are rounded to whole numbers. Any user edit sets `is_custom`, and the value stays until the user taps Reset to recommended.

7.2 Warning levels (calories)

Level	Condition	UI	Notification
Normal	Under 80% of target	Green progress	None
Warning	80% to 100%	Amber banner with a short suggestion	Once per day
Over limit	Above 100%	Red state with a suggestion	Once per day
Protein reminder	Protein under 60% of target at the reminder time	None	Once per day, if turned on

After every add, edit or delete: recompute totals, update Home, then check `alert_log` to decide whether to notify.

7.3 Chart

- Bar chart of daily calories with a horizontal line at the target.
- Bars over the target are red, bars at 80% to 100% amber, others green.
- Ranges: 7 days, 30 days, All (the retention window). For ranges over 60 days, show weekly averages so the chart stays readable.
- Under the chart: average calories per day, number of days over target, number of days logged.

7.4 Portion adjustment

When the user changes grams from g0 to g1, each nutrition value is multiplied by g1 / g0.

8. API design

There is one backend function. It is a Firebase callable function, so the app SDK attaches the logged-in user's token and the function receives the verified user id.

8.1 `analyzeMeal`

Request:

```
{
  "imageBase64": "<JPEG, long side at most 1024 px, about 300 KB or less>",
  "mealType": "lunch"
}
```

Success response:

```
{
  "scansRemaining": 3,
  "items": [
    { "name": "Roti", "grams": 80, "kcal": 240, "protein_g": 8, "carbs_g":
46, "fat_g": 3 },
    { "name": "Dal", "grams": 150, "kcal": 180, "protein_g": 10, "carbs_g":
24, "fat_g": 5 }
  ],
  "confidence": "medium"
}
```

8.2 Function steps

1. Reject if the user is not logged in.
2. Reject if the image is missing or larger than 1 MB.
3. In a Firestore transaction, read today's count (IST). If it is at or above the limit, reject with `LIMIT_REACHED`.
4. Call the LLM with the image and a fixed instruction: identify each food item on the plate, prefer Indian food names and common portions, estimate grams, return only JSON in the agreed schema, and return an empty list if there is no food.
5. Parse and validate the answer: schema matches, numbers are within sane ranges (for example, kcal per item between 0 and 2000), and the number of items is at most 10. Clamp or reject bad values.
6. If valid, add 1 to the count and return the response. A failed analysis does not use up a scan.
7. Discard the image. Never log image data.

8.3 Errors

Code	Meaning	App behaviour
UNAUTHENTICATED	Not logged in or token expired	Send the user to the login screen
LIMIT_REACHED	Daily scans used up	Show the limit message and offer manual add
NO_FOOD_DETECTED	The model found no food	Ask to retake the photo
BAD_IMAGE	Missing or too large	Retake the photo

Code	Meaning	App behaviour
AI_UNAVAILABLE	LLM error or timeout	Retry button and manual add
NETWORK	No connection	Retry button and manual add

9. Module design (app)

9.1 Layers

Layer	Folder	Content
UI	`lib/ui/`	Screens and widgets only. No calculations.
Logic	`lib/logic/`	Target calculator, alert evaluator, totals, retention cleaner. Plain Dart, easy to unit test.
Data	`lib/data/`	Drift database and DAOs, auth repository, scan repository (calls `analyzeMeal`)
Core	`lib/core/`	Theme, constants, error types, date helpers

9.2 Feature folders and ownership

Feature	Screens	Suggested owner
auth	Splash, Welcome, Sign up, Log in	Vishal
scan	Camera, Result	Vishal
backend	`analyzeMeal` function	Vishal
onboarding	Profile setup, Targets	Riya
home	Home dashboard, Edit entry	Riya
history	History and chart, Day detail	Riya
settings	Settings	Riya

Each owner reviews the other's pull requests.

9.3 Key components

Component	Responsibility
`TargetCalculator`	Profile to calories and macros (section 7.1)
`AlertEvaluator`	Totals and targets to a level: normal, warning or over
`TotalsService`	Sums entries for a date or a date range
`RetentionCleaner`	Deletes entries older than the chosen period
`ScanRepository`	Compresses the image, calls `analyzeMeal`, maps errors
`AuthRepository`	Sign up, log in, log out, session state
`NotificationService`	Schedules and sends local notifications, checks `alert_log`

10. UI and UX design

10.1 Design principles

- Fewest taps to log a meal: Home, Scan, shutter, Save.
- Today first: Home always shows today.
- Honest numbers: AI results are clearly labelled and always editable.
- Calm warnings: amber then red, always with one helpful suggestion.

10.2 Visual system

Item	Value
Style	Light theme, rounded cards, bold numbers
Background	#F7F9F3
Card	#EDF1E6
Text	#16231A, secondary #4F5A4C
Primary action	Deep green #1B7F36 with white text
Protein, carbs, fat	Coral #D9532B, blue #1C86C4, amber #B97A00

Item	Value
Warning, over limit	Amber #B97A00, red #C62828
Font	Bricolage Grotesque (fallback: system sans)
Corner radius	24 px for cards, 28 px for buttons

A dark theme is a V1 Should and follows the phone's system setting.

10.3 Screen inventory

ID	Screen	Purpose
S1	Splash	Check the session and route to Home or Welcome
S2	Welcome	Choose Log in or Sign up
S3	Sign up	Name, email, password
S4	Log in	Email, password, forgot password
S5	Profile setup	Goal, body details, targets review (3 steps)
S6	Home	Today's progress, meals and the Scan button
S7	Camera	Take or choose a meal photo
S8	Result	Review, adjust and save the AI estimate
S9	History	Calorie chart, range selector, summary, day list
S10	Day detail	Entries and totals for one day
S11	Settings	Profile, targets, retention, notifications, account

The Home, Scan, Onboarding and Result screens already have mockups on the shared canvas, in dark and light versions.

10.4 Navigation map

```
Splash -> (no session) Welcome -> Sign up / Log in -> Profile setup -> Home
Splash -> (session, profile exists) Home
Home (bottom bar): Home | History | [Scan] | Settings
Scan -> Camera -> Result -> Save -> Home
History -> Day detail -> Edit entry
```

10.5 Screen specifications

Screen	Main elements	States to design
Sign up	Name, email, password with show/hide, Create account, link to Log in	Field errors, weak password, email already used, loading
Log in	Email, password, Log in, Forgot password, Google button (Should)	Wrong password, no internet, loading
Profile setup	Step 1 goal cards. Step 2 age, gender, height, weight, activity. Step 3 recommended targets with Edit and Accept. Not-medical-advice note.	Invalid ranges disable Continue; edited values marked custom
Home	Date, calorie ring with kcal left, macro bars, warning banner, today's meals, scans left, bottom bar	Empty day, normal, warning, over limit
Camera	Live preview, shutter button, gallery button, scans-left chip, Add manually link	Permission explanation, permission denied, limit reached
Result	Photo thumbnail, AI-estimate chip, items with grams and kcal, macro chips, portion stepper, Rescan, Save to today	Loading (skeleton), success, several items, error with retry, no food found
History	Range tabs (7d, 30d, All), bar chart with target line, average and days-over-target summary, list of days	Empty, fewer days than range, chart for 6 months
Day detail	Date, totals versus target, entry list with edit and delete	Empty day
Settings	Profile, targets (recommended or custom, reset), retention 1/3/6 months, notifications and reminder time, units note, log out, delete data, delete account	Confirm dialogs for destructive actions

10.6 Copy and accessibility

- Always say "estimate" for AI results.
- Warning text examples: "You have used 85% of today's calories. Keep the next meal light." and "You are 120 kcal over today's target."

- Never rely on colour alone: pair colour with a label or icon.
- Support larger system text sizes without clipping.

11. Key flows

11.1 Sign up and setup

1. User opens the app. Splash finds no session and opens Welcome.
2. User signs up. Firebase creates the account and starts a session.
3. The app creates a local database file for this user id.
4. User completes Profile setup. Targets are calculated and saved.
5. Home opens showing zero.

11.2 Returning user

1. Splash finds a saved session and a local profile.
2. The retention cleaner runs.
3. Home opens. No login is needed, even offline.

11.3 Scan a meal

1. User taps Scan. The app shows the scans-left count.
2. The user takes a photo. The app compresses it.
3. The app calls `analyzeMeal`. The function checks the limit, calls the LLM and validates the result.
4. The Result screen shows the items. The user adjusts if needed.
5. User taps Save to today. Rows are written to `meal_entry`.
6. Totals are recomputed, Home refreshes and alerts are evaluated.

11.4 Limit alert

1. After a save, `AlertEvaluator` compares today's calories to the target.
2. If 80% or 100% is newly crossed and `alert_log` has no entry for that level today, Home shows the banner and a notification is sent if allowed.
3. The level is written to `alert_log`.

11.5 Change retention

1. User picks a new period in Settings.
2. If the period is shorter, the app asks for confirmation.

3. The cleaner deletes entries older than the cutoff, and the chart refreshes.

12. Security and privacy design

Area	Design
Secrets	The LLM key exists only as a Cloud Function secret. The app contains no keys.
Transport	HTTPS only
Authentication	Firebase tokens, verified automatically for callable functions
Abuse control	Per-user daily limit, 1 MB image cap, App Check enabled to reject fake clients (Should)
Firestore	Rules deny all client reads and writes
Passwords	Handled by Firebase. Never stored or logged by the app.
Photos	Held in memory during the request, never written to disk or logs on the server
Local data	Stored in the app's private storage, and excluded from device backups unless the user opts in
Deletion	Settings can delete all local data, and delete the account, which also removes the scan counters
Policy	Privacy policy and not-medical-advice statement are required before publishing

13. Error handling and edge cases

Situation	Behaviour
No internet at scan	Message with retry and Add manually
LLM timeout	Same as above, and the scan is not counted
Photo with no food	Ask to retake, scan is not counted
Limit reached	Message explaining it resets at midnight IST. Manual add stays available.

Situation	Behaviour
Camera permission denied	Explain why it is needed, link to system settings, gallery and manual add still work
Phone date changed manually	Calorie days follow the phone date. The scan limit follows the server day, so it cannot be gained by changing the date.
Session expired	Return to Log in, local data is kept
Different user logs in on the same phone	They get their own database file
App killed during save	Save runs in one database transaction, so there are no half-saved meals
Shortened retention period	Confirm, then delete

14. Test design

14.1 Strategy

Level	Scope	How
Unit	Target calculator, alert evaluator, totals, retention cleaner, portion maths	Automated Dart tests
Widget	Form validation, warning states, result card editing	Automated Flutter widget tests
Function	`analyzeMeal` limit, validation and errors	Firebase emulator with a mocked LLM
Integration	Sign up to first saved meal, scan to saved entry, day rollover	Automated on a device
Device	Camera and permissions on at least 3 phones, Android 10 to 14, including low-end	Manual
Usability	Time to log a meal, clarity of alerts	5 gym-goers with a short feedback form

Level	Scope	How
AI accuracy	About 30 known meals including Indian plates. Target: most single dishes within about 25% on calories	Manual comparison with label or food-table values

14.2 Sample test cases

ID	Requirement	Test	Expected
TC-01	FR-07	Male, 21, 175 cm, 70 kg, moderate, maintain	About 2,625 kcal, 126 g protein, 73 g fat, 366 g carbs
TC-02	FR-06	Age 5 or weight 0	Inline error, Continue disabled
TC-03	FR-03	Log in, close the app, reopen	Home opens without login
TC-04	FR-01	Sign up with an email already used	Clear error message
TC-05	FR-14	Change an item from 180 g to 90 g	All nutrition values halve
TC-06	FR-15, FR-20	Save a result	Home totals update immediately
TC-07	FR-17	Make scans up to the limit, then one more	Last request returns LIMIT_REACHED
TC-08	FR-17	Change the phone date, then scan	Limit still enforced by the server
TC-09	FR-12	Turn off internet and scan	Retry and manual options, no scan used
TC-10	FR-25	Target 2,000, log up to 1,600	Warning state at 80%
TC-11	FR-25	Push the total to 2,001	Over-limit state
TC-12	FR-26	Cross 80% twice in a day (delete, re-add)	Only one notification
TC-13	FR-19	Change the phone date to the next day	New day shows zero, old day is in history

ID	Requirement	Test	Expected
TC-14	FR-22	Set retention to 1 month with 2-month-old data	Confirmation, then old entries are deleted
TC-15	FR-23	Open the chart with 6 months of data	Weekly averages shown, no lag
TC-16	FR-09	Deny camera permission	Explanation and settings link, manual add works
TC-17	FR-11	Send a photo of a non-food object	NO_FOOD_DETECTED, scan not counted
TC-18	D10	Log out, log in as another user	Separate history

14.3 Defect levels and release criteria

Level	Meaning	Rule
Critical	Crash, data loss, wrong totals, login broken, camera unusable	Fix before any release
Major	A Must requirement does not work as specified	Fix before release
Minor	Cosmetic or small usability issue	Fix if time allows

Release is ready when all Must requirements pass, there are no open Critical or Major defects, and the app has been tested on at least 3 phones.

15. Deployment and release design

Item	Design
Environments	Two Firebase projects: dev and prod
Build	`flutter build appbundle` for the Play Store, with code shrinking on
Backend deploy	`firebase deploy --only functions` from the repository
Config	Daily limit stored in Firestore `config/limits`, changeable without an app update

Item	Design
Monitoring	Function logs (without image data), and a billing budget alert on the LLM and Firebase usage
Store requirements	Privacy policy link, Play Data Safety form (email and user id collected; photos processed and not stored), content rating, app icon and screenshots
Versioning	Semantic versions, a Git tag per release

16. Traceability

Feature	Requirements	Screens	Components	Tests
F1 Account	FR-01 to FR-08, FR-28	S1 to S5, S11	AuthRepository, TargetCalculator	TC-01 to TC-04, TC-18
F2 Photo analysis	FR-09 to FR-12	S7	ScanRepository, analyzeMeal	TC-09, TC-16, TC-17
F3 Result details	FR-13 to FR-16	S8	Result view, portion logic	TC-05, TC-06
F4 Scan limit	FR-17, FR-18	S6, S7	analyzeMeal, Firestore counter	TC-07, TC-08
F5 History, chart, alerts	FR-19 to FR-27	S6, S9, S10, S11	TotalsService, AlertEvaluator, RetentionCleaner, NotificationService	TC-10 to TC-15

17. Open points

#	Question	Default until decided
1	Exact daily scan limit	5 per day
2	Google sign-in in V1 or later	Later (Should)
3	Which LLM and model to use, based on accuracy and cost tests	Gemini, to be confirmed with the 30-meal test set

#	Question	Default until decided
4	Whether to offer optional cloud backup of history in V1.1	No, V2
5	Launch date and promotion	Not decided

18. Handover to development

Deliverables from this phase: this document, the UI mockups on the shared canvas (Onboarding, Home, Scan, Result in dark and light), the database schema (section 6), the API contract (section 8) and the test cases (section 14). The next phase starts with the project skeleton, the authentication flow and the `analyzeMeal` function with a mocked LLM, so both builders can work in parallel.